

Gate-Pinned State Memory

Exact, zero-context memory in a model's recurrent state, served by tang

DRAFT

CAM PEDERSEN · MUNICIPAL ROBOTICS

Abstract

A coding agent remembers by pasting text into its prompt, and pays for it on every turn. Divyansh Agrawal's qwenputer showed another way: a Gated DeltaNet head with its gates pinned is exact, addressable RAM, and a filesystem can be compiled into extra layers of a model without training[1]. We call the method Gate-Pinned State Memory (GPSM), and checked every link of it on our own stack.

tang's kernel stores and reads one-hot rows bitwise and stays exact over a 50,000-step free run, while no trained head qualifies: on GLM-5.3-Flash, none of 8,192 channels per layer holds its gate at exactly 1. On an RTX 3090 the author's model issued every store call itself and read its files back from a 14 KB store file in an empty conversation, with its text unchanged to within the noise floor. tang then ran the compiled layers bitwise in its reference math, emitted all 168 of their tensors bitwise from its own Rust compiler, and served them from a CUDA window inside the engine at about 0.9 ms per token, 30 to 40 times faster than the author's stack, without disturbing the model's own state. Two hoped-for shortcuts to instant prefill failed.

Of the approaches we surveyed, GPSM is the only one that recalls exactly, costs no context and needs no training at once. It moves an agent's working memory out of the prompt, turns exact capabilities into compiled modules rather than training runs, and lets a user hold their memory as a file instead of leaving it with a provider. Its limits are a fixed capacity of hundreds of files, a store tied to one model, and the open question of whether a model reaches for it at the right moments.

DRAFT

1 Motivation

A coding agent's memory today is text. Notes, summaries and retrieved chunks go into the prompt, are prefilled on every turn, and compete with code for the context window. On a 16 GB laptop running a 4B model, context is the scarcest thing there is. The alternatives in the literature each pay somewhere else. KV caches are exact but large and tied to one model[2]. Learned fixed-size states cost no context but forget[3,4]. Test-time training writes facts into weights, at the price of optimizer state and lost general ability[5].

Gated DeltaNet (GDN) updates a matrix state per head with the delta rule[6]:

$$S_t = g_t S_{t-1} + \beta_t k_t (v_t - S_{t-1}^{\top} k_t)^{\top}$$

With the decay $g = 1$, the write strength $\beta = 1$ and a one-hot key $k = e_i$, the update replaces row i of S with v and leaves every other row alone.



FIG 1 The delta rule, drawn. Sixteen rows stand for a 128 × 128 head state. A trained head decays every row and smears the write across all of them. A pinned head with a one-hot key replaces exactly one row and leaves the rest untouched.

A one-hot query reads row i back. Compile weights that produce those gates and keys, and the recurrent state is RAM. We call fixing the gates this way *gate pinning*, a head built this way a *pinned head*, and memory kept in pinned heads **Gate-Pinned State Memory** (GPSM). The gates are pinned, not the state: rows change when they are rewritten, and stay exactly as written until then. qwenputer[1] compiled a RISC-V CPU and a 512-file store into 12 extra layers in front of Qwen3.5 4B, without training. It reported a KL divergence to the base model of 6×10^{-4}, and recall of facts across chats 90 percent of the time, against 30 percent when the old chats were pasted back in.

tang is our inference engine. It runs Qwen3-Coder-Next, a mixture of experts with about 3B parameters active per token and a hybrid of 36 GDN layers and 12 attention layers, and already snapshots the fixed-size recurrent state to disk. Its GDN kernel is specified for the shapes of Flash-Next, another Qwen hybrid tang serves, with 16 key and 48 value heads per layer; Coder-Next and Qwen3.5 4B have 32 value heads. frog, our coding agent, talks to it. So the question was concrete: what of this transfers to an engine we own?

DRAFT

2 Exact RAM in tang's kernel

tang pins the GDN recurrence in a CPU specification that the CUDA kernel is verified against bitwise. We tested the claim there.

- **The gates reach exactly 1.** Gate pre-activations of $a + \mathrm{dt} = -30$ for the decay and $b = 40$ for the write strength round to exactly `1.0f`.
- **A write into an empty row stores v bitwise** and touches no other row of any of a Flash-Next layer's 48 value heads.
- **An overwrite stores** $\mathrm{fma}(1, v - s, s)$, which is exactly v only when $v - s$ is exact. On random floats, 8,643 overwrites were exact and 15,933 were off by one rounding. Stored values must be integers below 2^{24}.
- **A 50,000-token free run**, driving the kernel as a register machine, stayed bitwise exact on the CPU and on an RTX 3090.
- **One token with $g < 1$ ends exactness for good.** It scales every row by a non-power-of-two, and the integers turn fractional.

3 Trained heads cannot do it

So a pretrained head can be borrowed for storage only if its gate is bitwise 1 on every token. We measured the real gates of GLM-5.3-Flash, whose recurrent layers use a per-channel-gated delta rule, over 427 tokens and all 34 recurrent layers.

TABLE 1 Gates of a trained hybrid model. No channel qualifies as exact storage. Measured on GLM-5.3-Flash UD-Q4_K_XL, RTX 3090.

LAYERS	MEAN G	BEST CHANNEL MIN G	MEAN / MAX BETA	AT 1.0 EVERY TOKEN
Early (0–10)	0.51–0.68	0.999994	0.54–0.71 / ≤0.997	0 of 8,192
Mid (12–26)	0.73–0.87	0.999999	0.64–0.77 / ≤1.000	0 of 8,192
Late (28–44)	0.67–0.79	0.999997	0.55–0.84 / 1.000	0 of 8,192

A few channels are remarkably slow. Between 1 and 8 percent per layer have a mean gate above 0.999, and the best has a mean gate of $1 - 2 \times 10^{-7}$, a half-life from the gate alone of about three million tokens.

DRAFT

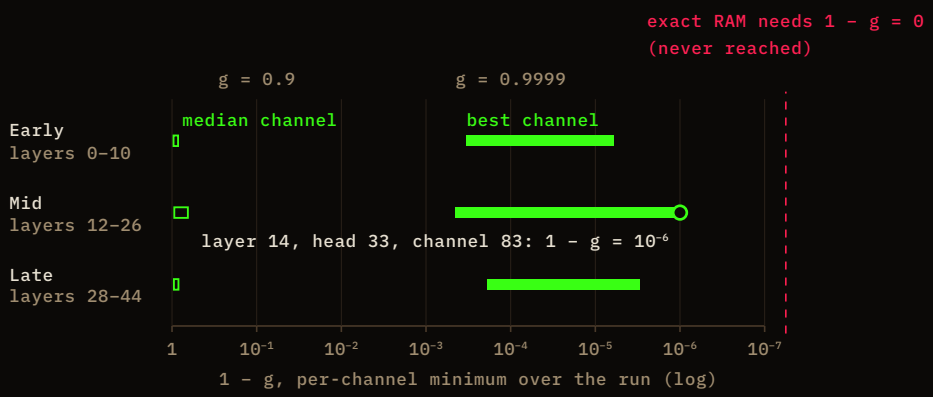


FIG 2 No channel reaches 1. Each bar is a range across the layers in its group: hollow for the median channel's minimum gate over the run, solid for the best channel's. GLM-5.3-Flash on an RTX 3090 over 427 tokens; per-layer data was not kept.

But they are analog registers, and they leak. Exact storage needs dedicated pinned heads with compiled gates, which is the qwenputer design and the basis of GPSM.

4 Two dead ends

The same structure suggests faster prefill. Writing context straight into state cannot work: a trained head can only read the state its own prefill would have produced, so an integer written into a row means nothing to it. We tested the two ideas that could work, and neither does.

Chunked kernels. tang steps the GDN recurrence one token at a time. A chunked tensor-core kernel has about 35 times the throughput on a 512-token window, but the recurrence is only 4.5 percent of a prefill window on an RTX 3090, and 5.5 percent on an H200. Prefill time lives in attention and the MoE layers.

Precomputed chunk maps. The delta rule is affine in S, so a chunk of text collapses to one matrix pair per head. If a chunk's keys and values did not depend on earlier context, prefill would be a few matrix products. We spliced context-free chunks into real GLM-5.3-Flash runs.

TABLE 2 Context-free chunk precomputation. KL is over the 32 tokens after each spliced chunk. About 0.1 nat per token: better than skipping the chunk, not lossless.

SPLICED CHUNK	STATE ERROR, EARLY / MID / LATE	KL, MEAN / MAX (NATS)	TOP-1 AGREE
4	0.34 / 0.65 / 0.54	0.095 / 0.80	30 / 32
8	0.30 / 0.62 / 0.46	0.103 / 1.13	27 / 32
12	0.44 / 0.98 / 0.60	0.111 / 0.89	31 / 32
Skip (control)	0.5–0.7	0.30–0.58 / 3.7–5.5	26–28 / 32

Mid and late layers depend on context, not position. Fixing the error means recomputing most of the depth, which is prefill again. Instant restore of *old* context is what tang already ships: a resumed 7,000-token session prefills in about 0.1 s from the snapshot disk tier.

DRAFT

5 GPSM, end to end

5.1 An engine-driven store

Figure 3 shows the whole path. frog keeps the store as a file on the user's disk and gives the model a `Memory` tool. tang serve loads the file into a session's pinned heads at the start and exports it after every write. Inside the engine, store operations run as their own windows through the compiled layers, apart from the host model's layers and KV cache.

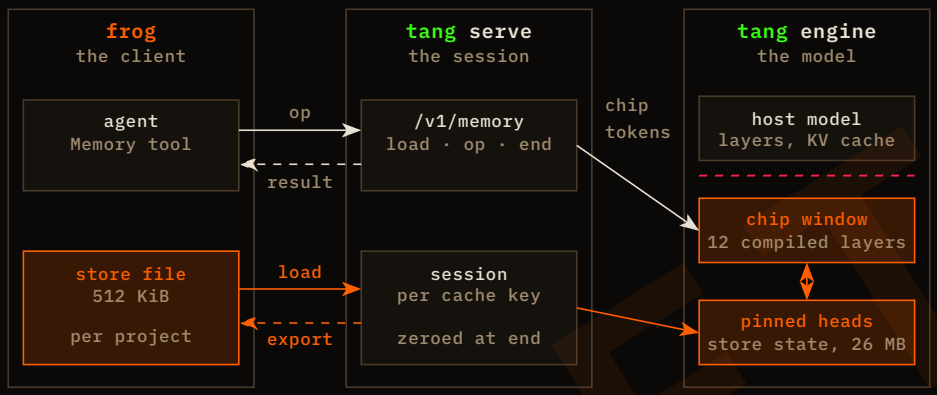


FIG 3 Architecture. Store traffic, in orange, never passes through the host model: operations run as chip tokens through the compiled layers, which read and write the pinned heads. The red line marks that separation. The store file moves only between the user's disk and a live session.

We first built the store without compiled layers: a key-value file store in dedicated GDN heads of one layer, written and read through the real kernel with one-hot keys and pinned gates. The layout follows the author's: a hashed directory with linear probing, plus content rows of token ids. Values stay below 2^{18}, tighter than the kernel's 2^{24}, because a read passes the output norm and a division, about eight roundings.

TABLE 3 Store capacity. The store takes 4 of a layer's 16 key heads and their value heads, a quarter of one GDN layer. The heads column gives the whole layer.

MODEL SHAPE	HEADS, K / V	DIRECTORY	CONTENT
Flash-Next	16 / 48, tiled	1,024 slots, 512 files	65,024 tokens · 768 KiB
Coder-Next, Qwen3.5 4B	16 / 32, grouped	512 slots, 256 files	65,024 tokens · 512 KiB

A 512-file fill and a 10,000-operation random workload, checked against a hash map, both pass. The store persists through the bytes tang's snapshot already carries. It also moves into a different conversation's state while leaving that conversation's own heads bitwise untouched.

5.2 The model drives it

Reproduction. We ran the author's code and checkpoint on an RTX 3090. In one conversation the model issued all nine store calls itself: three puts, a list, three gets, a delete and a list. Every answer was printed by the model, and 160 printed tokens matched the author's reference with no mismatches. We saved the store the author's way, as a 14 KB file of its occupied rows, and loaded it into an empty conversation.

The model listed the two surviving files and read both back exactly. KL to the base model was 6.9×10^{-4} against a noise floor of 6.7×10^{-4}, and bitwise identical with files loaded or not.

tang runs the compiled layers. tang's reference math ran the checkpoint's 12 compiled layers on the author's trace. All 126 tokens of a shorter put, get and list trace matched, and every store head's 128×128 state was bitwise equal after each operation.

tang compiles them. `tang-chip`, a Rust port of the author's compiler, emits the layers from the circuit graph: 3,753 dimensions, 164 heads and 47,532 units. The dimensions are live at different times, so the compiler packs them into far fewer slots of the residual stream. At Qwen3.5 4B shapes, all 168 tensors equal the checkpoint bitwise, zero padding included: 2.44 billion elements, 1.24 million of them nonzero. Retargeted to Coder-Next, whose hidden size widens from 2,048 to 3,968, the store replays the trace bitwise and no weight touches the host model's dimensions.

tang serves them. A chip window in the engine uploads the emitted layers as sparse blocks and runs store calls without touching the base layers, the LM head or the KV cache. On an RTX 3090 it ran with 0 mismatching layer steps against the CPU path. Inside a running engine, on a slot holding a prompt, store calls returned the stored ids. The slot's tokens and greedy continuation stayed identical to a twin slot that never ran them.

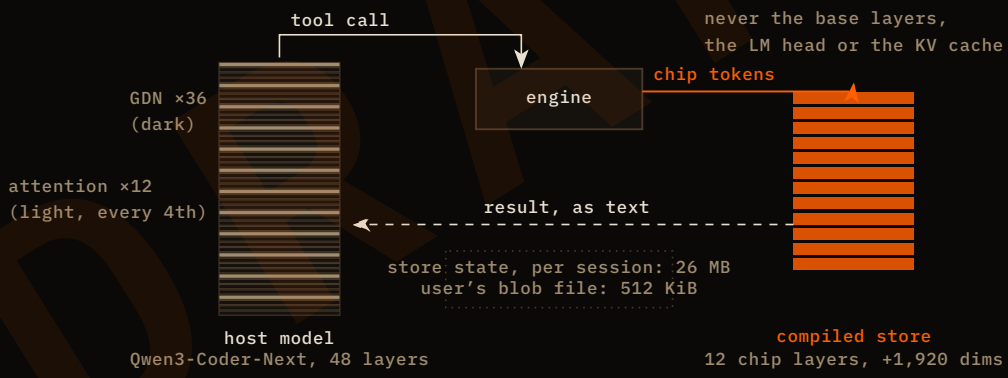


FIG 4 Where the store lives. The compiled layers sit beside the host model, not inside it. The host model emits the tool call; the engine runs it as chip tokens through the chip layers only, and the host sees the result as text.

TABLE 4 The chip window on an RTX 3090, with CUDA graphs. The author's PyTorch stack took 1.0–1.65 s per operation.

OPERATION	TOKENS	PER OP	PER TOKEN
Put	56	51 ms	0.92 ms
Get	43	40 ms	0.92 ms
List	27	25 ms	0.93 ms

DRAFT

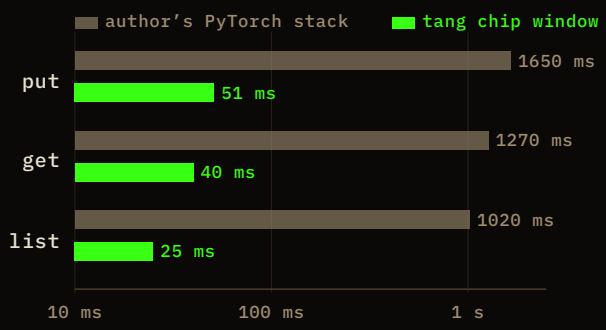


FIG 5 Latency per store operation, same RTX 3090, log scale. The author's time is mostly PyTorch fallback kernels, not the store.

The compiled layers take 13.75 MB of sparse weights and 26 MB of state per session. They cost about 332 million multiply-adds per token, roughly 8 percent of the 4B model.

5.3 Serving

`tang serve` loads a store per session, runs operations and exports it. An ephemeral mode zeroes the store when a session ends and never writes it to disk. frog loads the user's file at session start, saves after every write, and gives the model a `Memory` tool. `frog memory dump` prints exactly what is stored.

> ASIDE · PRIVACY
>
> The part of memory that persists becomes a file on the user's disk, 512 to 768 KiB. In a hosted session the provider sees it in plaintext while the session runs, and with the ephemeral mode keeps nothing afterwards. This is not encryption: the layout is known, so anyone holding the file can read it. The model can still repeat what it reads, so egress controls matter as before. What changes is where memory piles up: in a file you can read and delete, not with us.

6 Implications

6.1 Memory stops competing with code

Every fact a text memory recalls is a prompt token, prefilled on every turn and paid for in latency and in room for code. GPSM moves that working set out of the prompt. A store of 256 files on Coder-Next, or 512 on Flash-Next, about 65,000 tokens of content, sits in 512 to 768 KiB of state and costs nothing until the model asks for a file. A lookup costs a few decoded tokens, not a context's worth. On a laptop running a 4B model, where context is the scarce resource, that is the difference between remembering a project's conventions and having room for the file being edited.

6.2 Compiled, not trained

Nothing in GPSM is learned. The base weights are untouched, the store layers are emitted by a compiler from a program, and the model's text behaviour is unchanged to within the noise floor. That rules out the failure modes of parametric memory: no catastrophic forgetting, no optimizer state, no drift as memories accumulate[5]. It also makes memory a module: the same compiler that emits a file store emitted a RISC-V CPU, and on the author's benchmarks that CPU took Qwen3.5 4B from 55 to 97.5

DRAFT

percent on large multiplication without any tool call[4]. Capabilities that need exactness can ship as compiled layers rather than as training runs.

6.3 The engine owns the invariant

Exactness holds only while every gate in a pinned head stays bitwise at 1. A single ordinary token through a store head ends it. So GPSM is not something a model does alone; it is a contract between compiled weights and an engine that routes store traffic separately and audits the store before export. That favours owning the forward pass. tang can keep store tokens out of the base layers and the KV cache because it decides what runs where. A general-purpose serving stack would need the same routing added.

6.4 The primitive is spreading

Hybrid models with delta-rule layers are no longer a niche. Qwen3-Next and Qwen3.5 use Gated DeltaNet in three of every four layers[6], and GLM-5.3-Flash uses a per-channel variant. Every such model has layers where gate pinning gives exact storage, so the technique is available wherever these models run. What does not transfer is the store itself: it is compiled for one model's shapes and tokenizer, and a new model needs a new build.

6.5 What it does not change

GPSM does not make new context cheaper. Prefill still runs every token through the attention and MoE layers, and both of our attempts to shortcut it failed (§4). It does not replace the KV cache of the attention layers. It is a place to keep a small, exact working set across sessions, not a faster way to read a large one.

7 Use cases

None of these is measured yet beyond the store itself; they are where we expect GPSM to matter.

- **A coding agent's long-term memory.** Build commands, conventions, decisions and the user's preferences, kept per project and recalled exactly in the next session. This is what frog's `Memory` tool is for.
- **Exact facts where paraphrase is a bug.** Identifiers, configuration values, API signatures and file paths. Text memory recalls these through retrieval and generation; GPSM returns the stored token ids.
- **Small models on small machines.** A 4B model with a short context gains a store that costs no context. The less context a device can afford, the more this matters.
- **Hosted agents that don't keep your history.** The store travels as a file the user holds. A provider loads it for a session and, in the ephemeral mode, keeps nothing afterwards.
- **Handoff between sessions and agents.** A store file can be passed from one session or subagent to another and read back exactly, a shared scratchpad that never enters either prompt.

DRAFT

- **Exact computation.** The author's compiled CPU does arithmetic and runs MicroPython inside the forward pass. The same path could carry other exact routines a model gets wrong when it reasons in text.
- **Robots that remember their site.** Speculative for us today: a robot's language model keeping site-specific facts, such as where things are and what the last shift left unfinished, across sessions without growing its context.

8 Comparison

TABLE 5 Memory beyond the context window. GPSM is the only approach with exact recall that costs no context and no training. Sources:[2,4–9].

APPROACH	RECALL	CONTEXT PER TURN	TRAINING	BASE MODEL	WHAT PERSISTS
Text memory: RAG, MemGPT, Mem0	Lossy, retrieval-bound	Retrieved text, every turn	None	Unchanged	Plaintext notes, any model
Prefix KV reuse	Exact for the cached prefix	The whole prefix stays in attention	None	Unchanged	KV blocks, one model, large
Learned state: GDN, Titans	Lossy, decays	None	Pretraining	New architecture	Fixed-size state, one model
Test-time training	Lossy	None	Gradient steps at inference	Weights change	Fast weights, one model
Knowledge editing, LoRA memory	Facts only; free-form recall at zero	None	Edits or gradient steps	Weights change	Edited weights
GPSM	Exact	None	None, compiled	Unchanged (KL at noise floor)	A store file of 512–768 KiB, one model

Where GPSM wins. It is the only approach that is exact, costs no context and needs no training at once. Text memory pays context on every turn and recalls through retrieval and paraphrase[7,8,10]. KV reuse is exact, but the cached text still occupies attention, and its blocks grow with every token. Learned states and test-time training cost no context but forget or drift[3,4,9,11]. Knowledge editing and one-step LoRA memory perform poorly at agent recall: one study measured free-form recall at zero[5].

Where it loses. Capacity is fixed at 256 to 512 files; text memory scales to a corpus. A store is tied to one model and tokenizer; plaintext notes move anywhere. Text memory is legible without tools; a store needs `frog memory dump`. And GPSM is days old, while text memory ships in products.

They combine. The two are complements, not rivals. Text retrieval suits a large, loosely needed corpus such as documentation. GPSM suits a small working set that must come back exactly and is needed often. An agent can use both, and the comparison is the experiment we have not run yet.

DRAFT

The compiled-weights lineage runs back to Tracr, which compiled programs into transformers to study them[12]. What is new is that hybrid models have layers where compiled storage is exact.

9 Open questions

- **No cross-session benchmark.** The 90 percent recall figure (§1) is the author's own fact set. Nobody has run GPSM, or the author's store, on LongMemEval or LoCoMo.
- **Does the model reach for it?** The 4B model used the store when asked. Whether Coder-Next uses memory at the right moments in a long coding session is unmeasured, and it is the main risk.
- **No build on real Coder-Next weights yet.** It needs the circuit re-exported with Coder-Next's tokenizer and a machine that holds the weights.
- **Fixed capacity.** 256 to 512 files, and frog stores text one byte per id today, about four times the slots real tokens would use.
- **Compute on every token.** About 8 percent at 4B, less on a 3B-active mixture of experts.

10 Conclusion

A delta-rule head with its gates pinned is exact RAM. We confirmed that in tang's kernel, showed that no trained head reaches it on its own, and followed the idea from the author's checkpoint to our own compiler and a CUDA window inside our engine, each step bitwise. Two hopes did not survive: the same structure gives no shortcut through prefill, and trained heads cannot be borrowed for storage. What survives is Gate-Pinned State Memory: a small, exact store that lives beside a model's own state, costs no context, needs no training, leaves the model's text unchanged, and persists as a file its user holds. Whether models use it well over long sessions is the next thing to measure.

References

1. Agrawal, D. Qwenputer: a fully functioning, deterministic RISC-V CPU inside Qwen3.5. https://divagr.com/blog/qwenputer (2026).
2. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. https://arxiv.org/abs/2405.16444 (2024).
3. Stuffed Mamba: Oversized States Lead to the Inability to Forget. https://arxiv.org/abs/2410.07145 (2024).
4. Titans: Learning to Memorize at Test Time. https://arxiv.org/abs/2501.00663 (2025).
5. Beyond Perplexity: A Behavioral Evaluation Framework for Deployment-Memory Claims in LLM Test-Time Training. https://arxiv.org/abs/2607.00368 (2026).

DRAFT

6. Gated Delta Networks: Improving Mamba2 with Delta Rule. https://arxiv.org/abs/2412.06464 (2024).
7. MemGPT: Towards LLMs as Operating Systems. https://arxiv.org/abs/2310.08560 (2023).
8. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. https://arxiv.org/abs/2504.19413 (2025).
9. In-Place Test-Time Training. https://arxiv.org/abs/2604.06169 (2026).
10. A-MEM: Agentic Memory for LLM Agents. https://arxiv.org/abs/2502.12110 (2025).
11. Latent Context Compilation: Distilling Long Context into Compact Portable Memory. https://arxiv.org/abs/2602.21221 (2026).
12. Tracr: Compiled Transformers as a Laboratory for Interpretability. https://arxiv.org/abs/2301.05062 (2023).

DRAFT